Guion de presentación de Paso Firme para Fabiana y Rita

Material para la presentación o mesa redonda del 30 de octubre de 2026. Proyecto LUSOGIRLS de automatizaciones con n8n para la zapatería de Marisa. **Formato de ensayo: 5 minutos de demo, 5 de preguntas y 2 de feedback.** La daily del 28/10 indica que el formato definitivo de mesa redonda depende de la confirmación de Alex; este guion no acredita esa confirmación.

Las intervenciones están en español para poder ensayarlas y decirlas directamente. Las indicaciones de pantalla son acciones, no texto para leer. Fabiana y Rita tienen aproximadamente dos minutos y medio cada una durante la demo.

Preparación antes de ensayar

Estado contrastado el 29/10 en GitHub: `dev` está en `b416e2a`; PF-00 a PF-10 están exportados. PF-01 y PF-02 ya están integrados mediante las PR #37 y #38, y PF-03 mediante #36. Las diez historias Must están Done en Projects. HU-13/datáfono (#42), DOC-03/mapa (#41) y la entrega parcial de CI/HU-16 (#44) siguen en revisión; HU-14 está entregada en la PR #45, pendiente de revisión e integración. **Ensayar el recorrido completo con Rita en una única instancia antes de mostrarlo.** La integración en Git no acredita por sí sola la configuración de esa instancia.

Fabiana prepara: app, n8n, Mailpit, PF-00, abonos por correo, cierre diario, errores y bitácora. **Rita prepara:** confirmación de apartado, recibo de abono, cambio de mensaje al liquidar y sus exportaciones. **Las dos:** ensayar una vez con cronómetro y acordar qué nivel pueden demostrar realmente.

- [ ] Levantar la app y los servicios locales de correo y registro, con la configuración local ya preparada:

```
docker compose -f docker-compose.yml -f infra/automatizaciones.compose.yml up -d
```

- [ ] Elegir una sola instancia. El laboratorio existente usa app, n8n y Mailpit. La instancia habitual usa 8000/5678 y necesita su propio Mailpit en 8025. No mezclar la app de una instancia con las credenciales, rutas y correo de la otra. Las copias de aceptación se desactivan al terminar; configurar y ensayar expresamente la copia de demo.
- [ ] Confirmar los destinos reales de PF-00 para PF-01 y PF-02. Comprobar que la URL del webhook de la app corresponde al modo de ejecución elegido. En modo de prueba, volver a poner el listener a la escucha antes de cada evento.
- [ ] Seleccionar PF-08 en **Settings** → **Error workflow** de cada flujo de negocio y verificarlo en la instancia. Los JSON exportados de esta copia no traen esa asociación configurada. PF-08 no debe apuntar a sí mismo.
- [ ] Preparar un destinatario ficticio en Mailpit y comprobar creación, correo, abono final y segundo correo. Usar el mismo apartado durante todo el recorrido. Para enseñar estados por separado, están preparados los datos por API en la tienda aislada del puerto 28001; esa carga no envía correos.
- [ ] Revisar nombres de nodos y disposición de izquierda a derecha. Tener los JSON finales en `workflows/`.

- [ ] Buscar `key` y `token` en los JSON y revisar las coincidencias en privado: un nombre de campo o una referencia de credencial no equivale a una clave expuesta. Comprobar también nodos y prompts.
- [ ] Comprobar que `.env` está ignorado y que no está entre los archivos versionados. No abrir sus valores durante la presentación.
- [ ] Dejar abiertos AGENTS.md, las specs, la bitácora y la evidencia del cierre sin movimientos.

Comandos de revisión desde la raíz del proyecto:

```
rg -n -i 'key|token' --glob '*.json' .
git ls-files -- .env
git check-ignore .env
```

El segundo comando debe quedar sin salida; el tercero debe mostrar `.env`. Esto comprueba el estado local: revisar también los archivos de la rama que vais a enseñar en GitHub.

Reparto de los cinco minutos de demo

Tiempo	Quién habla	Qué muestra la otra
0:00–0:30	Fabiana	Rita deja la app en pantalla
0:30–1:35	Rita	Fabiana crea el apartado y abre el correo
1:35–2:15	Fabiana	Rita muestra PF-00 y la entrada recibida
2:15–3:20	Rita	Fabiana registra el abono final y abre el recibo
3:20–4:20	Fabiana	Rita muestra PF-08 y la bitácora
4:20–4:40	Rita	Fabiana muestra las carpetas de documentación
4:40–5:00	Fabiana	Rita vuelve a la vista general de los flujos

Fabiana abre la presentación

0:00–0:30. Pantalla: app de Paso Firme.

> Hola, somos Fabiana y Rita, del equipo LUSOGIRLS. Nuestro proyecto automatiza tareas de la zapatería Paso Firme, de Marisa. La tienda permite reservar unos zapatos e ir pagándolos mediante abonos. Con n8n buscamos que las confirmaciones, los recibos y los avisos lleguen sin tener que prepararlos a mano. Vamos a enseñar el recorrido de un apartado desde su creación hasta que queda pagado.

Rita presenta la creación del apartado

0:30–1:35. Fabiana rellena y guarda el formulario mientras Rita explica.

Datos ficticios para el ensayo y la demo:

Campo	Valor
Cliente	Cliente Demo Presentación
Email	demo.presentacion@example.test, solo para Mailpit
Producto	Zapatos de prueba
Talla	38
Precio	60,00 €
Abono inicial	20,00 €
Forma de pago	Efectivo

Usar Mailpit con este destinatario ficticio. Si la demo incorpora la rama de HU-14, marcar el permiso del cliente ficticio y comprobar la credencial del webhook antes de probar; esa ampliación requiere su revisión e integración propias.

> Vamos a crear un apartado de prueba. El precio es de sesenta euros y la clienta deja veinte euros como abono inicial. La aplicación guarda la operación y calcula cuánto queda pendiente y la fecha límite. Al guardar, envía el evento a n8n, que pone en marcha la confirmación.

Fabiana abre el correo que acaba de llegar y compara sus datos con la app.

> Aquí vemos la confirmación: el producto, lo abonado, los cuarenta euros pendientes y el plazo. Estos importes vienen de la aplicación. El flujo utiliza esos datos para preparar el mensaje. También podemos comprobar que este correo corresponde al apartado que acabamos de crear.

Fabiana explica el recorrido en n8n

1:35–2:15. Rita abre PF-00 y la ejecución correspondiente.

> Esta es nuestra entrada común de eventos, PF-00. El nodo Recibir eventos recibe el aviso de la app. Después validamos el evento y su destino, y el Switch lo dirige al flujo correspondiente. En este caso, apartado creado va a PF-01. Un abono registrado va a PF-02. Así tenemos una entrada común y cada automatización tiene su responsabilidad.

Rita señala `body.evento` y `body.datos` en la entrada, sin mostrar cabeceras sensibles.

> Aquí se ve de dónde salen los datos. Por ejemplo, el correo del cliente está en `body.datos.email`. Los nombres de los nodos nos ayudan a seguir el recorrido.

Rita muestra la liquidación y el cambio de mensaje

2:15–3:20. Fabiana vuelve al mismo apartado y registra un abono de 40,00 €. Si se usa el webhook de prueba, rearmar antes el listener.

> Ahora la clienta paga los cuarenta euros que faltaban. La aplicación actualiza el apartado y lo deja liquidado. El evento de abono lleva el importe de este pago y los datos actualizados del apartado. PF-02 utiliza esa información para preparar el recibo.

Fabiana muestra primero `pendiente = 0` y `estado = liquidado` en la app; después abre el nuevo correo.

> El mensaje cambia: ahora indica que ya puede recoger sus zapatos. Si hubiera pagado solo una parte, el recibo mostraría lo que todavía queda pendiente. Es la aplicación la que calcula el saldo y determina el estado. Aquí comprobamos que el correo refleja esos mismos valores.

Fabiana explica errores y uso del agente de IA

3:20–4:20. Rita muestra PF-08 y después la entrada B05 de la bitácora.

> Para los fallos técnicos tenemos PF-08, el flujo de errores. En cada flujo de negocio hay que comprobar su asociación en Settings. El aviso identifica el flujo y el paso que ha fallado, con un resumen que evita copiar información sensible.

> También utilizamos un agente de IA como apoyo al desarrollo. Le damos las reglas en AGENTS.md y los requisitos de cada flujo en las specs. En la bitácora registramos las decisiones, las correcciones y las pruebas.

> Un problema real apareció al evitar avisos duplicados: Postgres podía devolver un elemento aunque no se hubiera creado una nueva reserva. Añadimos una comprobación explícita antes de enviar el correo y repetimos la prueba. La propuesta del agente necesitó esa verificación para funcionar correctamente.

Rita explica cómo se revisa el trabajo

4:20–4:40. Fabiana muestra `specs/`, `workflows/` y `docs/ai-log.md`.

> Aquí están las especificaciones, los flujos exportados y la bitácora. Nuestro reparto permite que cada una trabaje en sus tareas y que la otra las revise. Para comprobar una funcionalidad miramos también su ejecución y el resultado, como acabamos de hacer con los correos.

Fabiana propone una mejora para producción

4:40–5:00. Pantalla: vista general del proyecto.

> Para llevarlo a la tienda necesitamos completar la revisión de seguridad y preparar el despliegue con acceso protegido, HTTPS y copias cuya restauración hayamos probado. La API ya usa una clave. La autenticación del webhook y el consentimiento por apartado están trabajándose en HU-14: mostraremos su estado real, sin confundir una prueba de laboratorio con una publicación. Ahora podéis revisar los flujos y hacernos preguntas.

Respuestas para los cinco minutos de auditoría

Son respuestas de apoyo: la otra pareja elige las preguntas. Responder en unos 30–40 segundos y enseñar el nodo, archivo o ejecución que lo demuestra. Si algo no está integrado en la instancia, decirlo.

Rita responde sobre funcionamiento

Pregunta: ¿Cómo sabéis que el mensaje cambia cuando queda liquidado?

> Comparamos el estado y el saldo de la app con el correo recibido. Con un pago parcial se informa del pendiente; con estado liquidado y saldo cero se añade el aviso de recogida. El evento trae esos valores y el flujo los utiliza.

Mostrar: el mismo apartado y sus dos correos. Como referencia, spec de confirmación y spec de recibo.

Rita responde sobre nombres y orden

Pregunta: ¿Se entiende el flujo sin que vosotras lo expliquéis?

> Usamos nombres que describen la acción, como Recibir eventos, Validar evento y destino y Distribuir eventos. Los códigos PF nos permiten relacionar cada automatización con su spec y su JSON. Podemos recorrerlo siguiendo las conexiones desde la entrada hasta el resultado.

Mostrar: PF-00 con el zoom suficiente para leer los nodos. Revisar antes su disposición visual.

Fabiana responde sobre seguridad

Pregunta: ¿Dónde están las claves? ¿El webhook está protegido?

> Las claves que usan los nodos están en las credenciales de la instancia; la configuración local de la app queda fuera de Git. La API usa X-API-Key. En dev b416e2a, el webhook todavía no exige autenticación. HU-14 coordina una cabecera distinta en la app y n8n y añade permiso de correo por apartado. Antes de afirmar que una instancia tiene esos controles, comprobamos su versión, credenciales y pruebas.

Mostrar: la referencia a la credencial, sin abrir su valor, y la comprobación de Git preparada. No afirmar que se ha revisado todo el repositorio si solo se ha mirado un archivo.

Fabiana responde sobre errores y casos límite

Pregunta: ¿Qué pasa si no hay pagos ese día o llega un correo de pago dudoso?

> En el cierre diario, cuando no hay ventas ni abonos, el resultado es cero y aparece Sin movimientos. Tenemos ese caso documentado en las ejecuciones 17 y 18 del laboratorio. Para un pago por correo dudoso, la regla de PF-03 es no registrar un abono y pedir revisión a Marisa. La aceptación depende de las validaciones y de los datos de la app.

Mostrar: evidencia del cierre y, para PF-03, la ejecución de la versión que vayáis a presentar. **Si PF-03 no está integrado**, completar: «Esa es la regla definida en nuestra spec; hoy no la estamos demostrando en esta instancia».

Si preguntan por un fallo del correo:

> El fallo queda visible. Si no sabemos si el mensaje llegó a enviarse, comprobamos la ejecución y el buzón antes de reintentar, para evitar duplicados.

Rita responde sobre eficiencia y la IA dentro del flujo

Pregunta: ¿Qué decide la IA y qué decide el flujo?

> En los abonos por correo, la IA extrae datos como ordenante, importe y concepto. Después el flujo valida los datos y consulta la API. Las reglas determinan si se puede registrar el pago. Los saldos del apartado vienen de la app. Para acceder a un dato usamos expresiones directas, como `{{ $json.body.datos.email }}`, cuando ese es el formato de entrada del nodo.

Mostrar: los nodos de PF-03 y su spec. La evidencia de #36 documenta diez escenarios con OpenRouter real del 28/10, incluida repetición sin duplicado. Es una prueba anterior, no la demo actual. La comparativa del 29/10 usa los mismos nueve correos: Dots3-Note produjo nueve respuestas correctas; LFM2.5 produjo tres y seis HTTP 429. No llamar errores de extracción a esos límites del proveedor. Diferenciar esta extracción del agente usado para desarrollar el proyecto.

Fabiana responde sobre el agente de desarrollo

Pregunta: ¿Para qué sirven AGENTS.md, las specs y la bitácora? ¿Qué error cometió el agente?

> AGENTS.md contiene el contexto y las reglas comunes. Las specs describen la entrada, el resultado esperado y los casos de prueba de cada flujo. La bitácora guarda cómo usamos la IA y qué corregimos. Un ejemplo es el elemento que devolvía Postgres cuando no había una nueva reserva: añadimos una comprobación antes del envío y verificamos que una repetición no mandara otro aviso.

Mostrar: bitácora B05 y evidencia de PF-08. Las pruebas citadas son del laboratorio del 22 de octubre, no de la demo actual.

Los dos minutos de feedback sobre nuestro proyecto

Primer minuto: escuchar lo mejor que ha visto la otra pareja. Rita toma nota del flujo, nodo o resultado mencionado.

Segundo minuto: escuchar su mejora concreta. Fabiana anota qué cambiar, dónde y cómo comprobarlo. Responder después de escuchar, adaptando esta frase:

> Gracias. Nos llevamos la observación sobre [punto concreto]. Vamos a revisar [flujo o nodo] y comprobar el cambio con [caso de prueba].

Si no sabéis responder a una pregunta:

> Ese detalle no lo tenemos confirmado. Podemos enseñaros cómo está ahora y lo apuntamos para revisarlo.

Si la demo falla, mirar la ejecución y explicar el punto del fallo. Podéis enseñar una prueba anterior, indicando claramente que pertenece al ensayo. La creación del apartado por sí sola no demuestra que haya llegado el correo.

Cuando nos toque auditar a la otra pareja

Fabiana pregunta por seguridad, errores y uso del agente. Rita pregunta por funcionamiento, nombres y eficiencia. Mientras una pregunta, la otra anota lo que ve. Elegir una pregunta por bloque y pedir que enseñen el flujo.

Bloque	Quién pregunta	Pregunta propuesta
Funcionamiento	Rita	¿Podéis crear un apartado y enseñarnos el correo? ¿Dónde cambia el mensaje cuando se liquida?
Nombres y orden	Rita	¿Podemos seguir el recorrido desde la entrada hasta el envío leyendo los nombres?
Seguridad	Fabiana	¿Podéis mostrar la referencia a las credenciales y cómo comprobáis que .env queda fuera de Git?
Errores y casos límite	Fabiana	¿Podéis enseñar qué ocurre en un cierre sin movimientos y dónde está asociado el flujo de errores?
Eficiencia	Rita	¿Qué datos extrae la IA y en qué nodo se decide si el pago se acepta?
Agente de IA	Fabiana	¿Qué propuesta del agente tuvisteis que corregir y dónde quedó registrada?

Para darles feedback, Rita comenta un punto fuerte y Fabiana propone una mejora. Usar observaciones concretas:

> Lo que mejor nos ha parecido es [flujo o nodo], porque vimos que [resultado observado]. Esto ayuda a [beneficio para la tienda].

> Para producción propondríamos [cambio concreto] en [punto del flujo]. Lo comprobaríamos con [prueba], esperando [resultado].

Plantilla del escrito para entregar al terminar

Completar después de la auditoría con lo que realmente haya ocurrido. Extensión orientativa: media página. Rita recoge las observaciones sobre la otra pareja; Fabiana añade el feedback recibido y los cambios acordados. Las dos revisan el texto antes de entregarlo.

Equipo: Fabiana y Rita, LUSOGIRLS. **Pareja auditada:** [nombres]. **Nivel observado:** [nivel respaldado por la checklist].

> Auditamos a [nombres], cuyo proyecto consistía en [objetivo y automatizaciones mostradas]. Durante la demo vimos [acción realizada] y comprobamos [resultado concreto en la app, el flujo o el correo].

> Lo mejor de su trabajo fue [decisión o parte del flujo]. En [nodo o ejecución] observamos que [evidencia]. Nos pareció útil porque [beneficio o problema que evita].
>
> Para llevarlo a producción proponemos [mejora concreta]. Se aplicaría en [punto del flujo] y se comprobaría mediante [prueba y resultado esperado].
>
> Sobre nuestro proyecto, la otra pareja destacó [punto fuerte] y nos señaló [observación concreta]. A raíz de ello, vamos a cambiar [acción acordada] en [flujo] y verificarlo con [caso de prueba].
>
> Aprendimos [aprendizaje específico] al ver cómo resolvieron [situación]. Nos puede servir para [aplicación concreta en nuestro proyecto].